CSA1717-ARTIFICIAL INTELLIGENCE

PRACTICAL LAB EXPERIMENTS

RAGAVI S – (192565027)

EXPERIMENT 6: MISSIONARIES CANNIBAL PROBLEM

Aim

To understand and implement a solution for a Missionaries Cannibal problem.

Objective

- Three missionaries and three cannibals must cross a river using a boat that carries at most two people at a time.
- On neither riverbank should cannibals ever outnumber missionaries whenever missionaries are present, or the missionaries get eaten.
- The goal is to find the shortest safe sequence of boat crossings that gets everyone across, solved using state-space search (BFS).

Algorithm

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Source Code

```
from collections import deque
def is_valid(state):
  """Check if a state is safe: cannibals never outnumber missionaries
  on either bank (when missionaries are present)."""
  m_left, c_left, boat, m_right, c_right = state
  if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:
    return False
  if m_left > 0 and c_left > m_left:
    return False
  if m_right > 0 and c_right > m_right:
    return False
  return True

def get_successors(state):
  """Generate all valid next states from the current state."""
  m_left, c_left, boat, m_right, c_right = state
  successors = []
  # Possible moves: (missionaries, cannibals) taken in the boat
  moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
  for m, c in moves:
    if boat == 'left':
      new_state = (m_left - m, c_left - c, 'right', m_right + m, c_right + c)
    else:
      new_state = (m_left + m, c_left + c, 'left', m_right - m, c_right - c)
```

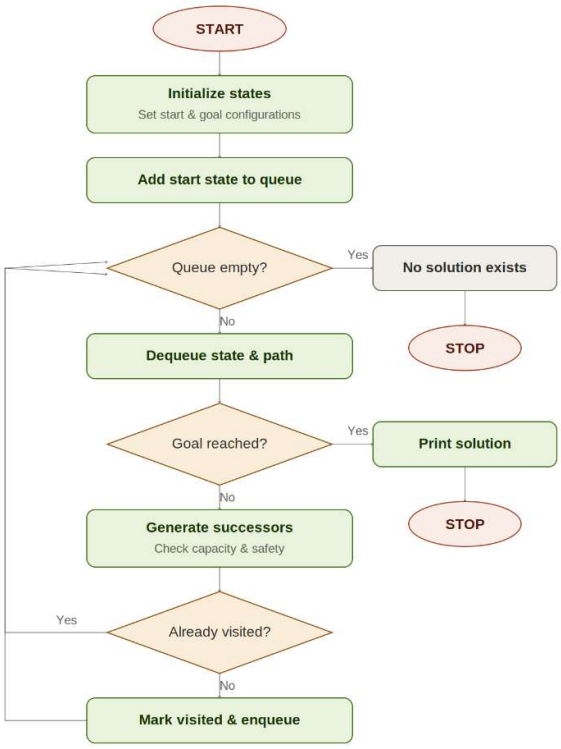
```
        if is_valid(new_state):
            successors.append((new_state, (m, c, boat)))
    return successors

def solve_missionaries_cannibals():
    """Solve using Breadth-First Search (BFS) to find the shortest safe path."""
    start_state = (3, 3, 'left', 0, 0)
    goal_state = (0, 0, 'right', 3, 3)
    frontier = deque([(start_state, [])])
    visited = {start_state}
    while frontier:
        current_state, path = frontier.popleft()
        if current_state == goal_state:
            return path
        for next_state, move in get_successors(current_state):
            if next_state not in visited:
                visited.add(next_state)
                frontier.append((next_state, path + [(next_state, move)]))
    return None
def print_solution(path):
    if not path:
        print("[-] No solution found.")
        return

print("[*] Initial State: 3 Missionaries, 3 Cannibals, Boat on LEFT bank\n")
    m_left, c_left, boat, m_right, c_right = 3, 3, 'left', 0, 0
    for step, (state, move) in enumerate(path, start=1):
```

```
    m, c, from_side = move
    direction = "LEFT -> RIGHT" if from_side == 'left' else "RIGHT -> LEFT"
    print(f"Step {step}: Move {m} Missionary(s) and {c} Cannibal(s) [{direction}]")
    m_left, c_left, boat, m_right, c_right = state
    print(f"    Left Bank: M={m_left}, C={c_left} | Right Bank: M={m_right}, C={c_right} | Boat: {boat.upper()}\n")
  print(f"[+] Goal reached in {len(path)} steps! All missionaries and cannibals crossed safely.")
if __name__ == "__main__":
  solution_path = solve_missionaries_cannibals()
  print_solution(solution_path)
```

Flowchart



Output

```
[*] Initial State: 3 Missionaries, 3 Cannibals, Boat on LEFT bank

Step 1: Move 0 Missionary(s) and 2 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=3, C=1 | Right Bank: M=0, C=2 | Boat: RIGHT

Step 2: Move 0 Missionary(s) and 1 Cannibal(s) [RIGHT -> LEFT]
    Left Bank: M=3, C=2 | Right Bank: M=0, C=1 | Boat: LEFT

Step 3: Move 0 Missionary(s) and 2 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=3, C=0 | Right Bank: M=0, C=3 | Boat: RIGHT

Step 4: Move 0 Missionary(s) and 1 Cannibal(s) [RIGHT -> LEFT]
    Left Bank: M=3, C=1 | Right Bank: M=0, C=2 | Boat: LEFT

Step 5: Move 2 Missionary(s) and 0 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=1, C=1 | Right Bank: M=2, C=2 | Boat: RIGHT

Step 6: Move 1 Missionary(s) and 1 Cannibal(s) [RIGHT -> LEFT]
    Left Bank: M=2, C=2 | Right Bank: M=1, C=1 | Boat: LEFT

Step 7: Move 2 Missionary(s) and 0 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=0, C=2 | Right Bank: M=3, C=1 | Boat: RIGHT

Step 8: Move 0 Missionary(s) and 1 Cannibal(s) [RIGHT -> LEFT]
    Left Bank: M=0, C=3 | Right Bank: M=3, C=0 | Boat: LEFT

Step 9: Move 0 Missionary(s) and 2 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=0, C=1 | Right Bank: M=3, C=2 | Boat: RIGHT

Step 10: Move 1 Missionary(s) and 0 Cannibal(s) [RIGHT -> LEFT]
    Left Bank: M=1, C=1 | Right Bank: M=2, C=2 | Boat: LEFT

Step 11: Move 1 Missionary(s) and 1 Cannibal(s) [LEFT -> RIGHT]
    Left Bank: M=0, C=0 | Right Bank: M=3, C=3 | Boat: RIGHT

[+] Goal reached in 11 steps! All missionaries and cannibals crossed safely.
>>>
```

Result

The Missionaries and Cannibals problem was successfully simulated, and the BFS-based search agent found a safe sequence of boat crossings that transported all missionaries and cannibals across the river without ever letting cannibals outnumber missionaries on either bank. Thus, the state-space search approach was implemented and verified correctly in Python, reaching the goal in 11 steps.